

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Kevan Patira (1BF24CS136)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Kevan Patira(**1BF24CS136**), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	
2	Implement Johnson Trotter algorithm to generate permutations.	
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	
9	Implement Fractional Knapsack using Greedy technique.	
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	
11	Implement "N-Queens Problem" using Backtracking.	

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design a technique to solve problems.

Lab program 1:

CODE:

```
#include <stdio.h>
#define MAX 10
int graph[MAX][MAX], indegree[MAX], queue[MAX];
int n;

void topologicalSort() {
    int front = 0, rear = 0;
    int count = 0, i, j;

    for (i = 0; i < n; i++) {
        indegree[i] = 0;
        for (j = 0; j < n; j++)
            indegree[i] += graph[j][i];
    }

    for (i = 0; i < n; i++)
        if (indegree[i] == 0)
            queue[rear++] = i;
    printf("Topological Ordering: ");
    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);
        count++;
        for (i = 0; i < n; i++)
            if (graph[v][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[rear++] = i;
            }
    }
    if (count != n) {
        printf("\nGraph contains a cycle. Topological sort not possible.\n");
    }
}

int main() {
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    topologicalSort();

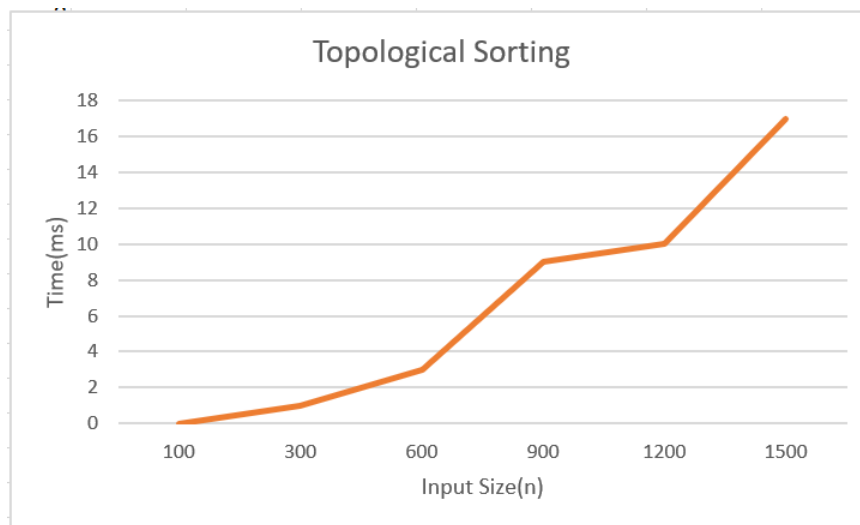
    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological Ordering: 0 1 2 3
-----
Process exited after 99.95 seconds with return value 0
Press any key to continue . . . |
```

```
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
1 0 0
Topological Ordering:
Graph contains a cycle. Topological sort not possible.
-----
Process exited after 22.42 seconds with return value 0
Press any key to continue . . . |
```

GRAPH:

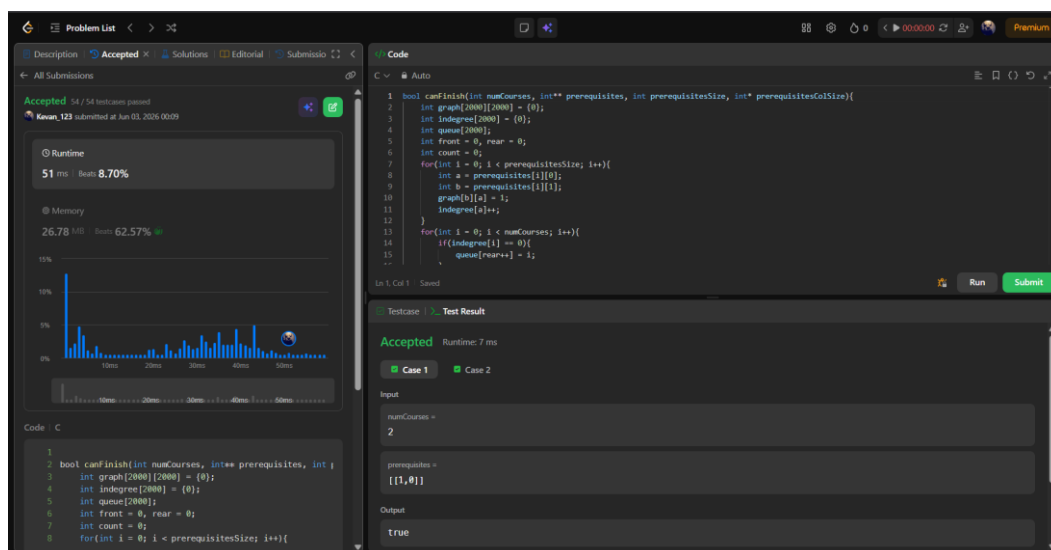


LEETCODE 207.Course Schedule

CODE:

```
#include <stdbool.h>
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize){
    int graph[2000][2000] = {0};
    int indegree[2000] = {0};
    int queue[2000];
    int front = 0, rear = 0;
    int count = 0;
    for(int i = 0; i < prerequisitesSize; i++){
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][a] = 1;
        indegree[a]++;
    }
    for(int i = 0; i < numCourses; i++){
        if(indegree[i] == 0){
            queue[rear++] = i;
        }
    }
    while(front < rear){
        int node = queue[front++];
        count++;
        for(int i = 0; i < numCourses; i++){
            if(graph[node][i] == 1){
                indegree[i]--;
                if(indegree[i] == 0){
                    queue[rear++] = i;
                }
            }
        }
    }
    return (count == numCourses);
}
```

OUTPUT:



Lab program 2:

CODE:

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

int mobile(int p[], int d[], int n) {
    int maxMobile = 0,i;

    for ( i = 0; i < n; i++) {

        if (d[i] == LEFT &&
            i > 0 &&
            p[i] > p[i - 1] &&
            p[i] > maxMobile) {

            maxMobile = p[i];
        }
        if (d[i] == RIGHT &&
            i < n - 1 &&
            p[i] > p[i + 1] &&
            p[i] > maxMobile) {
            maxMobile = p[i];
        }
    }
    return maxMobile;
}

int main() {

    int i,n;
    printf("Enter value of n: ");
    scanf("%d", &n);

    int p[10], d[10];

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        d[i] = LEFT;
    }

    while (1) {

        for ( i = 0; i < n; i++) {
            printf("%d ", p[i]);
        }
        printf("\n");
    }
}
```

```

int m = mobile(p, d, n);

if (m == 0)
    break;

int pos = -1;

for ( i = 0; i < n; i++) {
    if (p[i] == m) {
        pos = i;
        break;
    }
}
int next = pos + d[pos];
int temp = p[pos];
p[pos] = p[next];
p[next] = temp;
temp = d[pos];
d[pos] = d[next];
d[next] = temp;
for (i = 0; i < n; i++) {
    if (p[i] > m) {
        d[i] = -d[i];
    }
}
}
return 0;
}

```

OUTPUT:

```

Enter value of n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

-----
Process exited after 7.124 seconds with return value 0
Press any key to continue . . . |

```

Lab program 3:

CODE:

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>

void merge(int low, int mid, int high, int *a) {

    int i = low;
    int j = mid + 1;
    int k = low;

    int temp[high + 1];

    while (i <= mid && j <= high) {

        if (a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    // Copy back
    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

void mergeSort(int low, int high, int *a) {
    if (low >= high)
        return;

    int mid = (low + high) / 2;

    mergeSort(low, mid, a);
    mergeSort(mid + 1, high, a);

    merge(low, mid, high, a);
}

int main() {
    int n;
    printf("Enter size: ");
    scanf("%d", &n);

    int a[n];
    for (int i = 0; i < n; i++)
        a[i] = rand() % 1000;
```

```

clock_t stTime = clock();
mergeSort(0, n - 1, a);
clock_t endTime = clock();
double t =
    ((double)(endTime - stTime))
    / CLOCKS_PER_SEC;

printf("\nSorted Array:\n");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\n\nTime Taken: %.4f ms\n", t * 1000);

return 0;
}

```

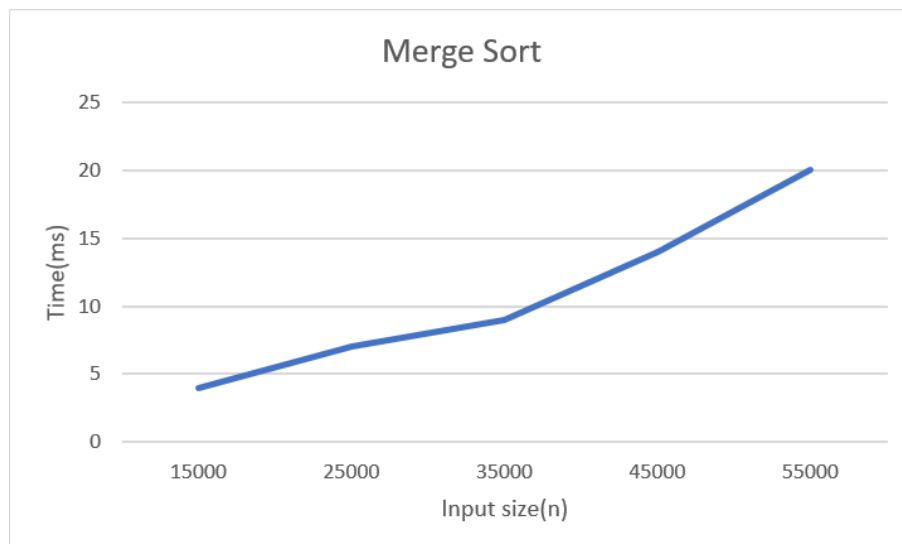
OUTPUT:

```

C:\Users\ADMIN\Downloads\test.exe
Enter the size of array: 10
Enter elements: 1 4 3 2 77 85 23 32 44 67
Sorted array: 1 2 3 4 23 32 44 67 77 85
Process returned 0 (0x0)   execution time : 16.382 s
Press any key to continue.

```

GRAPH:



LEETCODE 912.Sort an Array

CODE:

```
void merge(int arr[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    int temp[high + 1];
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)
        temp[k++] = arr[i++];
    while (j <= high)
        temp[k++] = arr[j++];
    for (int p = low; p <= high; p++)
        arr[p] = temp[p];
}

void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    mergeSort(nums, 0, numsSize - 1);
    *returnSize = numsSize;
    return nums;
}
```

OUTPUT:

The screenshot displays a code editor interface with a dark theme. On the left, a sidebar shows the 'Accepted' status of the submission, indicating that 21 out of 21 test cases passed. Below this, performance metrics are shown: Runtime is 56 ms (Beats 62.77%) and Memory is 59.28 MB (Beats 79.41%). A memory usage graph is also visible. The main editor area shows the C code for the merge sort algorithm. On the right, the 'Test Result' panel shows the 'Accepted' status for 'Case 1' and 'Case 2'. The input for Case 1 is [5, 2, 3, 1], and the output is [1, 2, 3, 5]. The expected output is also [1, 2, 3, 5].

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 void merge(int* nums, int left, int mid, int right) {
5
6     int n = right - left + 1;
7     int temp[n];
8 }
```

Lab program 4:

CODE:

```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
void quicksort(int low,int high,int arr[]);
int partition(int low,int high,int arr[]);
void quicksort(int low,int high,int arr[]){
    if(low<high){
        int p=partition(low,high,arr);
        quicksort(low,p-1,arr);
        quicksort(p+1,high,arr);
    }
}
int partition(int low,int high,int arr[]){
    int i=low;
    int j=high+1;
    int pivot=arr[low];
    int temp;
    while(i<j){
        do{
            i++;
        }while(i<=high && arr[i]<=pivot);
        do{
            j--;
        }while(arr[j]>pivot);
        if(i<j){
            temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
        }
    }
    temp=arr[j];
    arr[j]=pivot;
    arr[low]=temp;
    return j;
}
void main()
{
    clock_t start,end;
    double cpu_time_used;
    int n;
    printf("Enter number of elements:");
    scanf("%d",&n);
    int arr[n];
    printf("Enter elements:");
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    start=clock();
    quicksort(0,n-1,arr);
    end=clock();
}
```

```

cpu_time_used=(((double)(end-start))/CLOCKS_PER_SEC)*1000.0;
printf("Sorted array:");
for(int i=0;i<n;i++){
    printf("%d ",arr[i]);
}
printf("\nExecution time:%.5f ms",cpu_time_used);
}

```

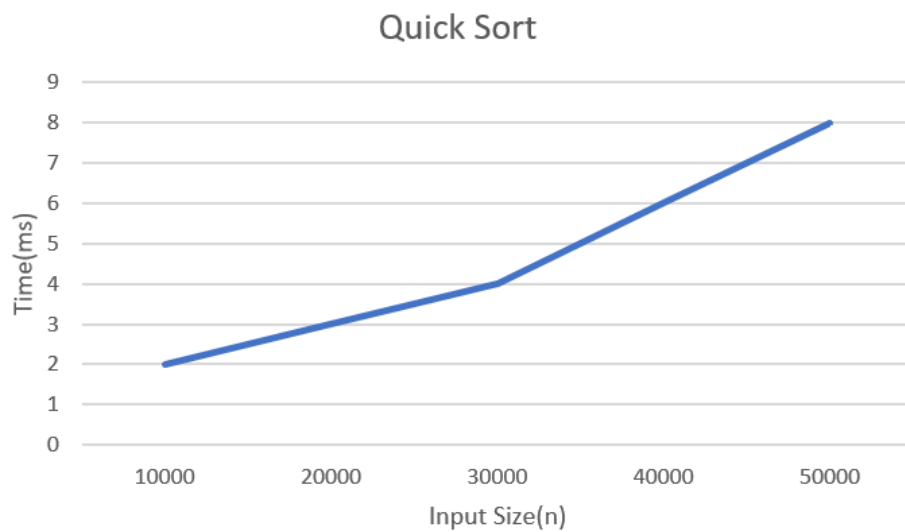
OUTPUT:

```

C:\Users\ADMIN\Downloads\test.exe
Enter number of elements:6
Enter elements:1 4 3 6 8 9
Sorted array:1 3 4 6 8 9
Execution time:0.00000 ms
Process returned 26 (0x1A)    execution time : 10.832 s
Press any key to continue.

```

GRAPH:



LEETCODE 977.Squares of sorted array

CODE:

```
int* sortedSquares(int* nums, int numsSize, int* returnSize) {
    int *arr = (int*)malloc(numsSize * sizeof(int));
    int left = 0;
    int right = numsSize - 1;
    int idx = numsSize - 1;
    while(left <= right) {

        int leftSq = nums[left] * nums[left];
        int rightSq = nums[right] * nums[right];

        if(leftSq > rightSq) {
            arr[idx--] = leftSq;
            left++;
        }
        else {
            arr[idx--] = rightSq;
            right--;
        }
    }
    *returnSize = numsSize;
    return arr;
}
```

OUTPUT:

The screenshot displays a coding competition interface with three main panels. The left panel shows submission statistics: 'Accepted 137 / 137 testcases passed', 'Kavan, 123 submitted at Jun 03, 2026 00:30', and performance metrics: 'Runtime: 0 ms, Beats: 100.00%', 'Memory: 25.41 MB, Beats: 47.14%'. A memory usage graph is also visible. The middle panel shows the C code for the 'sortedSquares' function. The right panel shows the 'Testcase' and 'Test Result' section, indicating 'Accepted' status with a runtime of 0 ms. It lists two cases, with Case 1 showing input '[-4,-1,0,3,10]' and output '[0,1,9,16,100]', matching the expected output.

```
1 /**
2  * Note: The returned array must be malloced,
3  * assume caller calls free().
4  */
5  int* sortedSquares(int* nums, int numsSize, int* returnSize) {
6      int *arr = (int*)malloc(numsSize * sizeof(int));
7      int left = 0;
8      int right = numsSize - 1;
9      int idx = numsSize - 1;
10     while(left <= right) {
11
12         int leftSq = nums[left] * nums[left];
13         int rightSq = nums[right] * nums[right];
14
15         if(leftSq > rightSq) {
16             arr[idx--] = leftSq;
17             left++;
18         }
19         else {
20             arr[idx--] = rightSq;
21             right--;
22         }
23     }
24     *returnSize = numsSize;
25     return arr;
26 }
```

Testcase: Accepted Runtime: 0 ms

Case 1 Case 2

Input: nums = [-4,-1,0,3,10]

Output: [0,1,9,16,100]

Expected: [0,1,9,16,100]

Lab program 5:

CODE:

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i;
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int n,i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d integer elements:\n", n);

    for ( i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    clock_t start = clock();
    heapSort(arr, n);
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```

printf("\nSorted elements:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
printf("\nTime taken: %.8f seconds\n", time_taken);
return 0;
}

```

OUTPUT:

C:\Users\ADMIN\Downloads\test.exe

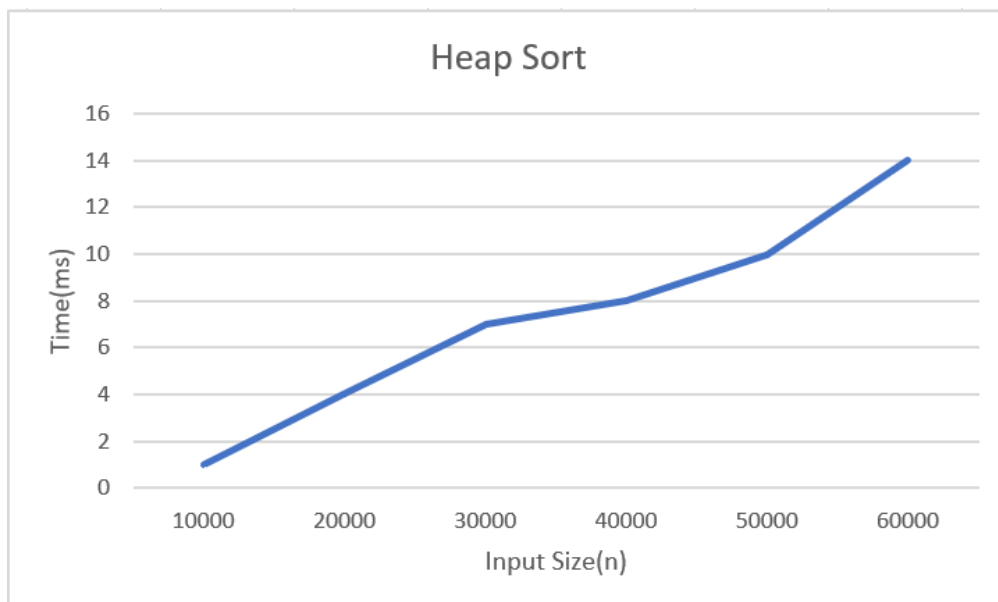
```

Enter number of elements:6
Enter the elements:
5 4 3 6 7 2

Elements after heap sort:2 3 4 5 6 7
Input Size:6 Execution Time:0.00000
Process returned 36 (0x24)   execution time : 6.256 s
Press any key to continue.

```

GRAPH:



Lab program 6:

CODE:

```
#include <stdio.h>
int max(int a , int b){
    return (a>b)?a:b;
}
int knapsack(int W, int wt[], int val[], int n){
    int dp[n+1][W+1], i,w;
    for( i=0;i<=n;i++)
        for( w=0;w<=W;w++)
            if(i==0 || w==0) dp[i][w]=0;
            else if(wt[i-1]<=w) dp[i][w]=max(val[i-1]+dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else dp[i][w]= dp[i-1][w];
    for( i=0;i<=n;i++){
        for( w=0;w<=W;w++)
            printf("%d\t",dp[i][w]);
        printf("\n");
    }
    return dp[n][W];
}

void main(){
    int n, W,i;
    printf("Enter number of items:");
    scanf("%d",&n);
    printf("Enter max capacity:");
    scanf("%d",&W);
    int val[n];
    int wt[n];
    printf("Enter wt and value for %d items:\n",n);
    for(i=0;i<n;i++)
        scanf("%d %d",&wt[i], &val[i]);
    printf("Result = %d", knapsack(W, wt, val ,n));
}
```

OUTPUT:

```

Enter number of items:3
Enter max capacity:50
Enter wt and value for 3 items:
10 60
20 100
30 120
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      60      60      60      60      60      6
0      60      60      60      60      60      60      60      60      60      60      60      60      60      60      6
0      60      60      60      60      60      60      60      60      60      60      60      60      60      60      6
0      60      60      60      60      60      0      0      0      0      60      60      60      60      60      6
0      60      60      60      60      100      100      100      100      100      100      100      100      100      100      1
60      160      160      160      160      160      160      160      160      160      160      160      160      160      160      1
60      160      160      160      160      160      0      0      0      0      60      60      60      60      60      6
0      0      0      0      0      0      0      0      0      0      60      60      60      60      60      6
0      60      60      60      60      100      100      100      100      100      100      100      100      100      100      1
60      160      160      160      160      160      160      160      160      160      180      180      180      180      180      1
80      180      180      180      180      220
Result = 220
-----
Process exited after 40.48 seconds with return value 12
Press any key to continue . . .|

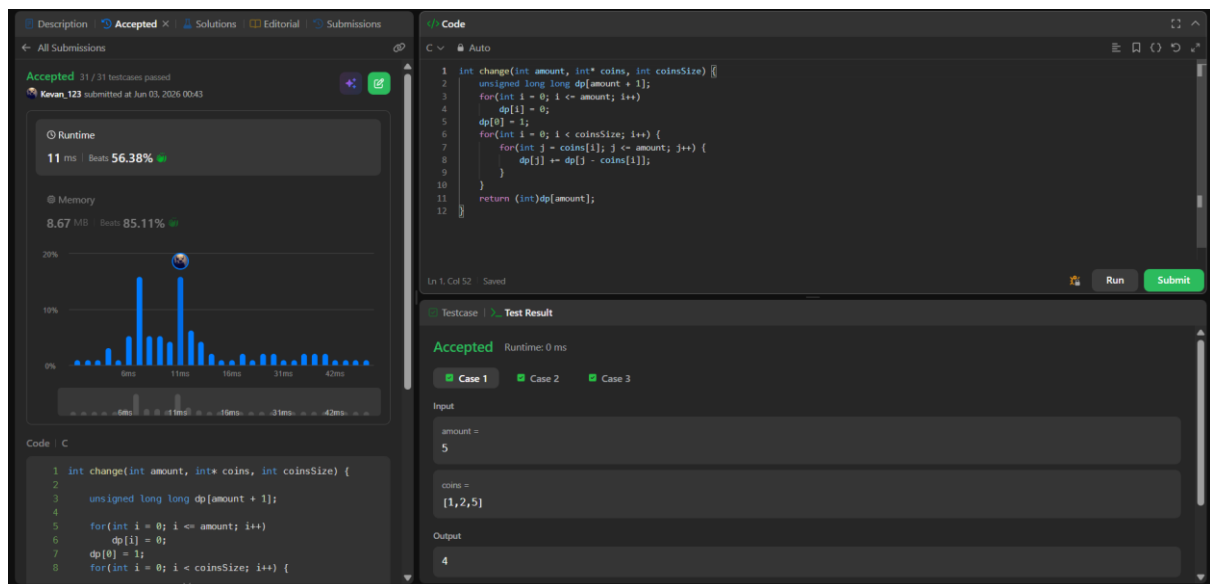
```

LEETCODE 518.Coin Change II

CODE:

```
int change(int amount, int* coins, int coinSize) {
    unsigned long long dp[amount + 1];
    for (int i = 0; i <= amount; i++)
        dp[i] = 0;
    dp[0] = 1;
    for (int i = 0; i < coinSize; i++) {
        for (int j = coins[i]; j <= amount; j++) {
            dp[j] += dp[j - coins[i]];
        }
    }
    return (int)dp[amount];
}
```

OUTPUT:



Lab program 7:

CODE:

```
#include <stdio.h>
#define N 10
#define INFI 99999
int min(int a, int b){
    return (a<b)? a: b;
}
void floyd(int d[N][N], int n){
    for (int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    printf("\nUpdated Path :\n");
    for (int i = 0; i < n; i++){
        for(int j = 0; j < n; j++)
            if (d[i][j] == INFI) printf("INF\t");
            else printf("%d\t", d[i][j]);
        printf("\n");
    }
}

void main(){
    int path[N][N];
    int n;
    printf("Enter number of nodes:");
    scanf("%d",&n);
    printf("Enter distance matrix\n(enter -1 for infinity):\n");
    for (int i=0;i<n;i++)
        for(int j=0;j<n;j++){
            int a;
            scanf("%d",&a);
            if(a==-1) path[i][j] = INFI;
            else path[i][j]= a;
        }
    floyd(path, n);
}
```

OUTPUT:

```
Enter number of nodes:4
Enter distance matrix
(enter -1 for infinity):
0 3 -1 8
-1 0 2 -1
-1 -1 0 1
-1 -1 -1 0

Updated Path :
0      3      5      6
INF     0      2      3
INF     INF     0      1
INF     INF     INF     0

-----
Process exited after 65.05 seconds with return value 4
Press any key to continue . . . |
```

```
Enter number of nodes:4
Enter distance matrix
(enter -1 for infinity):
0 5 10 -1
5 0 -1 3
10 -1 0 1
-1 3 1 0

Updated Path :
0      5      9      8
5      0      4      3
9      4      0      1
8      3      1      0

-----
Process exited after 34.87 seconds with return value 4
Press any key to continue . . . |
```

LEETCODE 783.Minimum Distance Between BST Nodes

CODE:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */
int minDiff = 100000;
struct TreeNode* prev = NULL;
void inorder(struct TreeNode* root) {
    if(root == NULL)
        return;
    inorder(root->left);
    if(prev != NULL) {
        int diff = root->val - prev->val;

        if(diff < minDiff)
            minDiff = diff;
    }
    prev = root;
    inorder(root->right);
}
int minDiffInBST(struct TreeNode* root) {
    minDiff = 100000;
    prev = NULL;
    inorder(root);
    return minDiff;
}
```

OUTPUT:

The screenshot displays a code editor interface with the following components:

- Top Bar:** Navigation tabs for Description, Accepted, Solutions, Editorial, and Submissions.
- Left Panel:**
 - Accepted:** 49 / 49 testcases passed. Submission by Kavan, 123 submitted at Jun 03, 2025 00:46.
 - Runtime:** 0 ms, Beats 100.00%.
 - Memory:** 9.52 MB, Beats 42.74%.
 - Code:** A snippet of the C++ code is visible.
- Right Panel:**
 - Code:** The full C++ code is displayed, matching the code provided in the previous block.
 - Testcase 1 / 2, Test Result:** Shows the test result for Case 1 and Case 2.
 - Accepted:** Runtime: 0 ms.
 - Case 1:** Input: root = [4, 2, 6, 1, 3], Output: 1, Expected: 1.

Lab program 8:

8(a)**CODE:**

```
#include <stdio.h>
#define MAX 10
#define INF 999
int main(){
    int cost[MAX][MAX];
    int visited[MAX] = {0};
    int n, i, j;
    int min, a, b;
    int edges = 0, minCost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    visited[0] = 1;
    printf("Edges in MST:\n");
    while(edges < n - 1){
        min = INF;
        for(i = 0; i < n; i++){
            if(visited[i]){
                for(j = 0; j < n; j++){
                    if(!visited[j] && cost[i][j] < min){
                        min = cost[i][j];
                        a = i;
                        b = j;
                    }
                }
            }
        }
        printf("%d - %d = %d\n", a, b, min);
        minCost += min;
        visited[b] = 1;
        edges++;
    }
    printf("Minimum Cost = %d\n", minCost);
    return 0;
}
```

OUTPUT:

```
Enter the number of vertices: 4
```

```
Enter the adjacency matrix:
```

```
0 4 6 5
```

```
4 0 2 3
```

```
6 2 0 7
```

```
5 3 7 0
```

```
0 - 1 : 4
```

```
1 - 2 : 2
```

```
1 - 3 : 3
```

```
Minimum Cost = 9
```

```
=== Code Exited With Errors ===
```

8(b)
CODE:

```

#include <stdio.h>
#define MAX 10
#define INF 999
int parent[MAX];
int find(int x){
    while(parent[x] != x)
        x = parent[x];
    return x;
}
void unionSet(int a, int b){
    parent[find(a)] = find(b);
}
int main(){
    int cost[MAX][MAX];
    int n, i, j;
    int min, a, b;
    int edges = 0, minCost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    for(i = 0; i < n; i++){
        parent[i] = i;
    }
    printf("Edges in MST:\n");
    while(edges < n - 1){
        min = INF;
        for(i = 0; i < n; i++){
            for(j = i + 1; j < n; j++){
                if(cost[i][j] < min){
                    min = cost[i][j];
                    a = i;
                    b = j;
                }
            }
        }
        if(find(a) != find(b)){
            printf("%d - %d = %d\n", a, b, min);
            minCost += min;
            unionSet(a, b);
            edges++;
        }
        cost[a][b] = cost[b][a] = INF;
    }
    printf("Minimum Cost = %d\n", minCost);
    return 0;
}

```

OUTPUT:

```
Enter the number of vertices: 4
Enter the adjacency matrix:
0 4 6 5
4 0 2 3
6 2 0 7
5 3 7 0
1 - 2 : 2
1 - 3 : 3
0 - 1 : 4
Minimum Cost = 9

-----
Process exited after 1934 seconds with return value 0
Press any key to continue . . . |
```

Lab program 9:

CODE:

```
#include<stdio.h>
struct object{
    float profit;
    float weight;
    float pw;
};
void sort(struct object o[],int n){
    struct object temp;
    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++){
            if(o[j].pw<o[j+1].pw){
                temp=o[j];
                o[j]=o[j+1];
                o[j+1]=temp;
            }
        }
    }
}
int main(){
    int n;
    printf("Enter number of objects:");
    scanf("%d",&n);
    struct object o[n];
    printf("Enter the profit and weight for:\n");
    for(int i=0;i<n;i++){
        printf("Object %d:",i);
        scanf("%f %f",&o[i].profit,&o[i].weight);
        o[i].pw=o[i].profit/o[i].weight;
    }
    sort(o,n);
    int size;
    printf("Enter the size of the knapsack:");
    scanf("%d",&size);
    float total_profit=0;
    int rm=size;
    int i=0;
    while(rm!=0){
        if(o[i].weight<rm){
            total_profit+=o[i].profit;
            rm-=o[i].weight;
        }
        else{
            total_profit+=((o[i].profit*rm)/o[i].weight);
            rm=0;
        }
        i++;
    }
    printf("%-12s %-12s %-12s %-12s\n", "Object", "Profit", "Weight", "Ratio");
    for(int i = 0; i < n; i++){
        printf("%-12d %-12.2f %-12.2f %-12.2f\n",
            i, o[i].profit, o[i].weight, o[i].pw);
    }
}
```

```
    printf("\nMaximum profit: %.2f\n", total_profit);  
    return 0;  
}
```

OUTPUT:

```
Enter number of items: 3  
Enter weight, profit and name respectively:  
10 60 A  
20 100 B  
30 120 C  
Enter capacity: 50  
  
Selected items: A B C(66.67%)  
Maximum Profit = 240.00  
  
-----  
Process exited after 35.44 seconds with return value 0  
Press any key to continue . . . |
```

LEETCODE 455.Assign Cookies

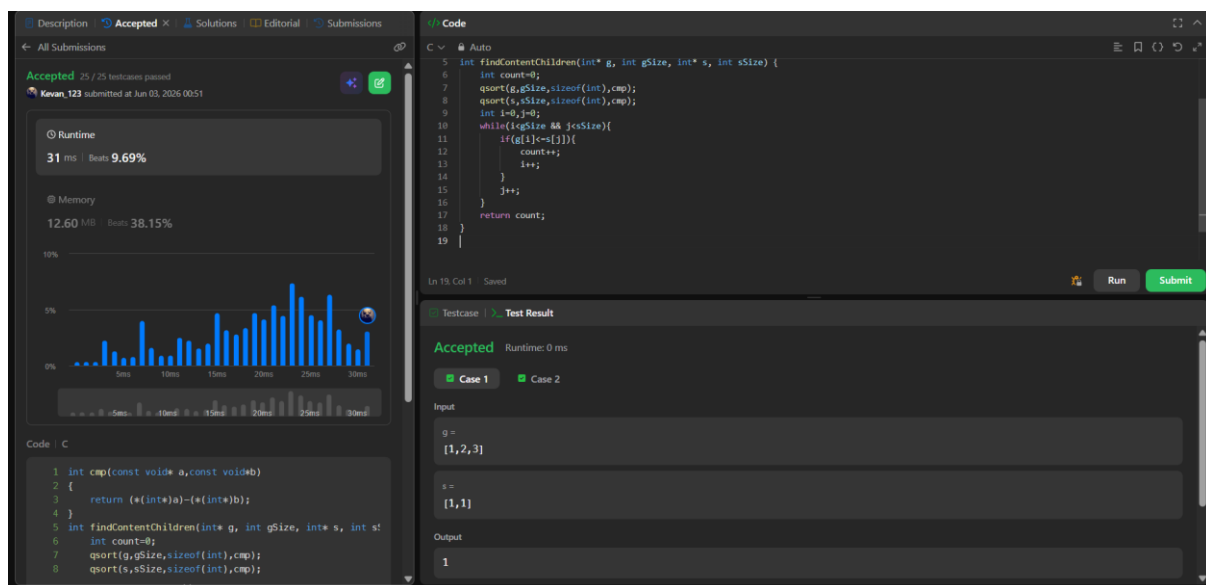
CODE:

```

int cmp(const void* a,const void*b)
{
    return (*(int*)a)-(*(int*)b);
}
int findContentChildren(int* g, int gSize, int* s, int sSize) {
    int count=0;
    qsort(g,gSize,sizeof(int),cmp);
    qsort(s,sSize,sizeof(int),cmp);
    int i=0,j=0;
    while(i<gSize && j<sSize){
        if(g[i]<=s[j]){
            count++;
            i++;
        }
        j++;
    }
    return count;
}

```

OUTPUT:



Lab program 10:

CODE:

```

#include<stdio.h>
#include<limits.h>
void p(int parent[],int j){
    if(parent[j]==j){
        printf(" Path:%d ",j);
        return;
    }
    p(parent,parent[j]);
    printf("%d ",j);
}
void main(){
    int n;
    printf("Enter the number of vertices:");
    scanf("%d",&n);
    int a[n][n];
    printf("Enter the weights and enter 0 for no edges:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&a[i][j]);
        }
    }
    int distance[n];
    int visited[n];
    int parent[n];
    int source;
    printf("Enter the source vertex:");
    scanf("%d",&source);
    for(int i=0;i<n;i++){
        distance[i]=(i!=source && a[source][i]==0)?INT_MAX:a[source][i];
        parent[i]=source;
        visited[i]=0;
    }
    distance[source]=0;
    visited[source]=1;
    for(int i=1;i<n;i++){
        int min=INT_MAX;
        int u=-1;
        for(int j=0;j<n;j++){
            if(!visited[j] && distance[j]<min){
                min=distance[j];
                u=j;
            }
        }
        if(u==-1) break;
        visited[u]=1;
        for(int v=0;v<n;v++){
            if(a[u][v]!=0 && !visited[v]){
                if(distance[u]+a[u][v]<distance[v]){
                    distance[v]=distance[u]+a[u][v];
                    parent[v]=u;
                }
            }
        }
    }
}

```

```

    }
    printf("The distances are:\n");
    for(int i=0;i<n;i++){
        printf("%d to %d",source,i);
        if(distance[i]==INT_MAX) printf(":No path");
        else{
            p(parent,i);
            printf(":%d",distance[i]);
        }
        printf("\n");
    }
}
}

```

OUTPUT:

```

Enter number of vertex:5
Enter weights:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Vertex Distance from Source :
0      0
1      10
2      50
3      30
4      60
-----
Process exited after 81.38 seconds with return value 5
Press any key to continue . . . |

```

```

Enter number of vertex:4
Enter weights:
0 4 2 0
4 0 0 5
2 0 0 1
0 5 1 0
Vertex Distance from Source :
0      0
1      4
2      2
3      3
-----
Process exited after 30.11 seconds with return value 4
Press any key to continue . . . |

```

Lab program 11:

CODE:

```

#include <stdio.h>
#include <stdlib.h>

int x[20];

int place(int k, int i) {
    for (int j = 1; j < k; j++) {
        if (x[j] == i || abs(j - k) == abs(x[j] - i))
            return 0;
    }
    return 1;
}

void NQueens(int k, int n) {
    for (int i = 1; i <= n; i++) {
        if (place(k, i)) {
            x[k] = i;

            if (k == n) {
                // Print solution as a binary matrix
                for (int row = 1; row <= n; row++) {
                    for (int col = 1; col <= n; col++) {
                        if (x[row] == col)
                            printf("1 ");
                        else
                            printf("0 ");
                    }
                    printf("\n");
                }
                printf("\n");
            }
            else {
                NQueens(k + 1, n);
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    NQueens(1, n);
    return 0;
}

```

OUTPUT:

Enter number of queens: 4

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0